Phase 3 — Continuous Batching Scheduler

Goal of this phase: build a scheduler that runs **one decode step across a whole batch of in-flight requests**, admitting new requests and evicting finished ones at step boundaries — the mechanism that separates a toy server from a real inference engine. Integrated with the Phase 2 KV cache (each sequence carries its own cache state inside the batch).

---

1. Static vs continuous batching (the key talking point)

Static batching: collect N requests, run them as one batch until *all* finish, then take the next batch. Problem: generations have different lengths. A batch of 8 where one request wants 200 tokens and seven want 10 tokens keeps the whole batch (and the GPU slots) busy for 200 steps while seven slots sit idle after step 10. The GPU is reserved but doing nothing useful — **head-of-line blocking**.

Continuous (in-flight) batching: the batch is dynamic. Each decode step, finished sequences are **evicted** and waiting requests are **admitted** into the freed slots — mid-flight, without waiting for the rest of the batch. The GPU stays full of *useful* work. For variable-length generation (i.e. all real workloads) this dramatically raises GPU utilization and throughput.

Measured on my engine (MPS, Qwen2.5-0.5B), aggregate tokens/sec:

concurrency	naive	batched	speedup
1	35.1	49.5	1.4×
4	51.9	74.1	1.4×
16	50.1	84.7	1.7×

The speedup **grows with concurrency** — the signature of batching. (Naive throughput is flat under load because requests are serialized; batched climbs.) On a real GPU with a larger model the gap is far wider — MPS kernel-launch and Python per-step overhead cap the win here.

---

2. What I built

- **`engine/batched.py` — `ContinuousBatchingEngine`:**
- A **single scheduler thread** owns the model and the batched cache. HTTP handler threads only enqueue a request and block on an `Event` until it finishes — so forward passes never race.
- **Admission:** a waiting request is **prefilled** alone (its prompt → its own KV cache + first token), then its cache row is **left-padded and merged** into the running batched cache.
- **Decode step:** one batched `model.forward` feeds every running sequence's next token, with per-row `position_ids` (true RoPE positions despite padding), a per-row `attention_mask`

(masking the left-pad), and a shared `cache_position` (all rows write the next slot).
- **Eviction:** sequences hitting EOS or `max_tokens` are removed from the batch and the cache via `DynamicCache.batch_select_indices`, and their freed slots are refilled next iteration.
- **Server integration:** `/generate` takes an `engine` field (`"naive"` | `"batched"`), so the same API and request schema serve both — the benchmark harness (Phase 4) can hit them identically.
- **Tests (`tests/test_batched.py`):** concurrent different-length requests produce **byte-identical** tokens to lone greedy decoding; batched aggregate throughput beats naive under concurrency.

Verification (MPS): 14/14 tests pass. Batched output is identical to single-sequence decoding; throughput exceeds naive at every concurrency level.

---

3. The hard part: batching per-sequence KV caches

Sequences join at different times and have different lengths, but a batched forward needs one cache tensor of shape `[B, H, L, D]` — the same `L` for every row. My solution:

- **Left-pad** every row to the batch's max length `L`. Padded positions hold zero K/V and are masked out by the attention mask, so they never affect outputs.
- Each row keeps its **true position** via `position_ids` (RoPE rotates Q/K by absolute position; the physical cache slot and the logical position differ for padded rows, and that's fine because RoPE uses `position_ids`, not the slot index).
- **Admission** pads the new row (or grows existing rows if the new prompt is longer) before concatenating along the batch dimension. **Eviction** drops rows with `batch_select_indices`.

This is exactly the inefficiency **PagedAttention (vLLM, Phase 5)** removes: instead of one contiguous padded block per sequence, it stores the cache in fixed-size **pages** and uses an indirection table, so there's no padding waste and no copying on admit/evict. My engine makes the problem visible; vLLM's design is the answer to it.

---

4. Design decisions & tradeoffs

Decision	Why / tradeoff
Single scheduler thread owns the model	Serializes all GPU access (no concurrent forward passes), and centralizes admit/decode/evict. Simple and correct. Tradeoff: one Python thread is the orchestration bottleneck — fine here, not how a multi-GPU server would scale.
Prefill new requests one at a time	Simpler admission. Tradeoff: prefills aren't batched together, so a burst of arrivals prefills serially. vLLM uses chunked/mixed prefill+decode batches to avoid this.

Decision	Why / tradeoff
Left-padding + masking for ragged lengths	Correct and uses HF's stock attention path. Tradeoff: padded slots waste compute and the cache isn't compacted on eviction — the exact waste PagedAttention eliminates.
`max_batch_size` cap (default 8)	Bounds memory (KV cache grows with batch × length). Under memory pressure the right move is to cap the batch and queue the rest — which is what the `waiting` queue does.
Greedy decoding	Deterministic, so batched output is provably identical to lone decoding — the correctness guarantee the tests assert.

Under memory pressure / full batch: new requests wait in the queue rather than OOM the GPU. A production engine would add preemption (evict a running sequence's cache to host memory and resume later) — vLLM does this; I bound batch size instead, and name the tradeoff.

---

5. A real bug I fixed

Under 16 concurrent first-time requests, the model loader crashed with "*Cannot copy out of meta tensor.*" Cause: `@lru_cache` caches the **result**, not the **execution** — so 16 threads all missed the cache simultaneously and raced on materializing the lazily-loaded ("meta device") weights. Fix: a `threading.Lock` around the load so the first concurrent callers serialize; later calls hit the cache. Lesson: caching ≠ thread-safety; the first concurrent miss is the dangerous one.

---

6. Interview Q&A

Q: Static vs continuous batching — why does continuous win? A: Static batching holds every slot until the longest sequence in the batch finishes, so short requests leave the GPU idle (head-of-line blocking). Continuous batching evicts finished sequences and admits waiting ones every step, keeping the batch full of useful work. For variable-length output — every real workload — that's a large utilization and throughput gain.

Q: How do you batch sequences of different lengths with one KV cache tensor? A: Left-pad all rows to the batch's max length, mask the padding in attention so it contributes nothing, and track each row's true position via `position_ids` so RoPE stays correct. Admission pads and concatenates a new row; eviction selects the surviving rows.

Q: What happens when the batch is full? A: New requests sit in a waiting queue and are admitted as slots free up (on eviction). This bounds KV-cache memory. A more advanced engine adds preemption — spilling a running sequence's cache to host memory and resuming it later — to stay responsive under heavy pressure.

Q: Where does your engine lose to vLLM? A: Padding waste and cache copying. I keep one contiguous, left-padded cache block per sequence, so padded slots waste compute and

admit/evict shuffles memory. PagedAttention stores the cache in fixed-size pages with an indirection table — no padding, no copying, near-zero fragmentation — which lets it pack far more concurrent sequences into the same memory. That's the throughput gap.

Q: Why one scheduler thread instead of one thread per request? A: The GPU does one forward pass at a time; concurrent threads calling the model would serialize on the device anyway and risk races. A single scheduler thread does the batched step and lets every in-flight request share that one pass — that sharing *is* the throughput win. Request threads just enqueue and wait.

Q: How do you guarantee batched output equals single-request output? A: Greedy decoding is deterministic, and correct masking + positions mean padding can't leak into a sequence's attention. The test runs four different-length prompts concurrently and asserts each produces byte-identical tokens to decoding it alone.

Q: Why did concurrent startup crash, and how did you fix it? A: `lru_cache` memoizes the return value but doesn't serialize the function body, so simultaneous first calls all executed the load and raced on meta-tensor materialization. I added a lock around loading; the first callers serialize and everyone after hits the cache.

---

7. One-line recall

> *"I built a continuous-batching scheduler: a single thread runs one batched decode step across all in-flight requests, admitting and evicting at step boundaries, with per-sequence KV caches merged by left-padding and correct per-row positions. It produces byte-identical output to lone decoding and beats the naive baseline's throughput, with the gap widening under concurrency — and I can explain precisely where vLLM's PagedAttention pulls ahead and why."*